LiDAR Sensing and Object Control in the Stretch Simulator

Robotics and Cyber-Physical Systems
School of Sciences and Engineering
Tecnológico de Monterrey

Overview

This document introduces two capabilities of the `stretch_toolkit` simulator: reading and visualising LiDAR data, and manipulating free-floating objects in the scene at runtime. It also explains how to extend the default scene with real-world 3D-scanned objects sourced from a public dataset.

Both features are demonstrated through ready-to-run scripts (`lidar_demo.py` and `object_control_demo.py` that follow the same boilerplate structure used throughout the course.

1 LiDAR Demo

1.1 What the Demo Does

The Stretch robot model includes a simulated 2D LiDAR sensor mounted on the base. It produces 360 range measurements per scan—one per degree—each reporting the distance to the nearest obstacle in that direction. `lidar_demo.py` reads these measurements every frame, drives the robot with the teleop provider, and renders a live top-down occupancy map in a separate window.

Run the demo with:

```
uv run lidar_demo.py
```

Press `Q` or `Escape` in the visualisation window to quit.

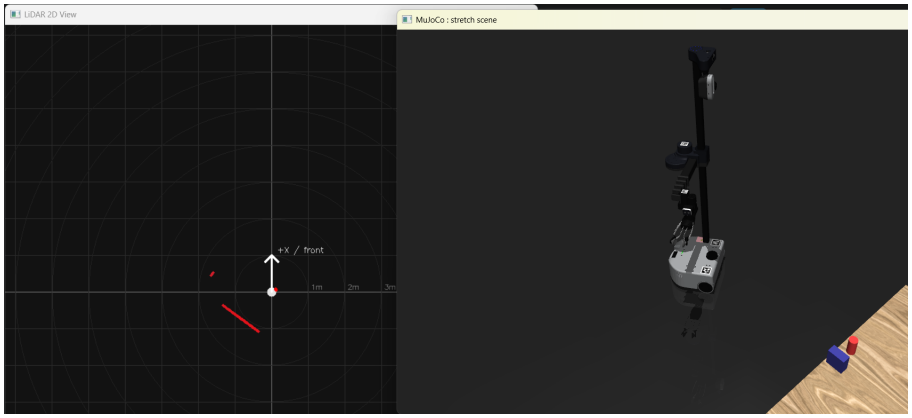


Figure 1: LiDAR demo: top-down 2D occupancy map rendered in real time. The robot is at the centre; detected obstacles appear as white dots.

1.2 How It Works

The complete source is shown below.

```python
from stretch_toolkit import controller, BACKEND_NAME, LidarPlotter,
    TeleopProvider
import time

def main():
    print(f"\n=== Running on {BACKEND_NAME} backend ===\n")
    print("Displaying LiDAR scan. Press Q or Escape to quit.\n")

    plotter = LidarPlotter()
    tp = TeleopProvider(is_stretch_env=False)
    while True:
        ranges = controller.get_lidar_ranges()
        velocities = tp.get_normalized_velocities()
        controller.set_velocities(velocities)
        plotter.update(ranges)

        if plotter.should_quit():
            break

    plotter.close()

if __name__ == '__main__':
```

```
22      main()
```

Key API calls

- `controller.get_lidar_ranges()` — returns a 360-element `numpy` array of distances in metres. Rays that did not hit any geometry contain `numpy.inf`.
- `LidarPlotter()` — creates an OpenCV window. Its `update(ranges)` method redraws the map each frame. Rays are drawn outward from the centre; shorter rays (closer obstacles) produce points nearer to the origin.
- `plotter.should_quit()` — returns `True` if the user pressed `Q` or `Escape` in the OpenCV window. Without this call the window would not respond to key presses.
- `plotter.close()` — destroys the OpenCV window cleanly.

Backend transparency `controller.get_lidar_ranges()` is defined on the base `JointController` class. On the physical robot it reads from the actual sensor; in simulation it samples the MuJoCo ray-cast sensor. The rest of the demo code is identical for both backends.

2 Object Control Demo

2.1 What the Demo Does

The simulator scene contains bodies declared with a `<freejoint/>` in the scene XML. These bodies are not attached to the robot or any fixture—they can be moved freely in six degrees of freedom at runtime. `object_control_demo.py` enumerates all such bodies, lets the user cycle through them with the keyboard or gamepad, and moves the selected body using a second input layer.

Run the demo with:

```
uv run object_control_demo.py
```

Press `Ctrl+C` in the terminal to quit.

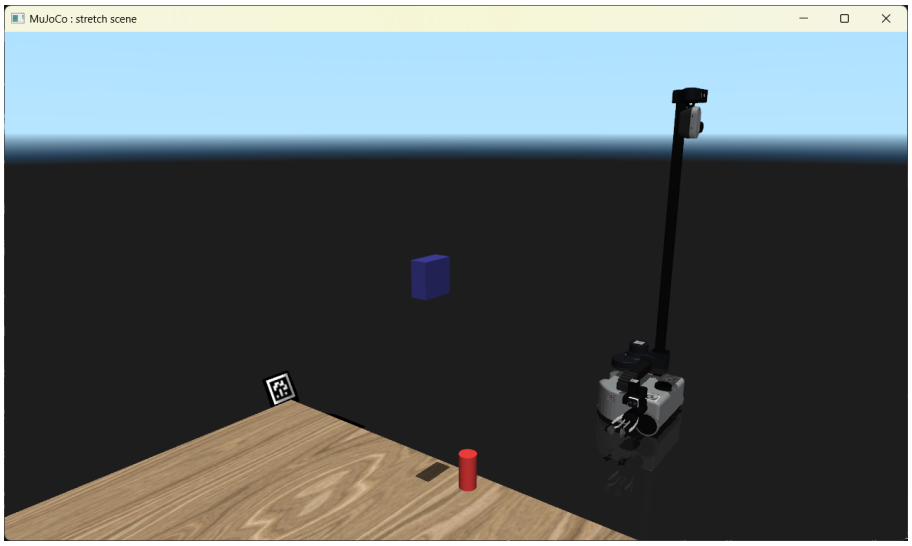


Figure 2: Object control demo: the selected object (highlighted in the MuJoCo viewer) is moved interactively. The terminal prints the current pose whenever the selection changes.

2.2 Controls

The demo has two modes: **Object** (default) and **Teleop**. Press `Tab` or the gamepad `START` button to switch between them. The current mode is printed in the terminal whenever it changes.

Key / Button	Mode	Action
t / START	Both	Toggle Teleop ↔ Object mode
m / RB	Object	Select next object
n / LB	Object	Select previous object
z / DPAD UP	Object	Move up (+z)
x / DPAD DOWN	Object	Move down (−z)
w / s	Object	Move forward / backward (+y / −y)
a / d	Object	Move left / right (−x / +x)
i / k	Object	Pitch + / −
j / l	Object	Yaw + / −
u / o	Object	Roll + / −
g / Y	Object	Gravity **OFF** — object floats
f / B	Object	Gravity **ON** — object falls
Standard teleop keys	Teleop	Drive the robot base

2.3 How It Works

```python
from stretch_toolkit import controller, BACKEND_NAME,
    ObjectControlProvider, TeleopProvider
import stretch_toolkit.input as inp
import time

MOVE_STEP = 0.02   # metres per tick
ROT_STEP  = 0.05   # radians per tick (~3 deg)
obj_ctrl  = ObjectControlProvider(move_step=MOVE_STEP, rot_step=
    ROT_STEP)
tp        = TeleopProvider(is_stretch_env=False)

def main():
    objects = controller.list_scene_objects()
    selected = 0
    mode = "object"

    while True:
        # --- Toggle mode (Tab on keyboard, START on gamepad) --------
        if inp.rising_edge("t", "START"):
            mode = "teleop" if mode == "object" else "object"
            print(f"[Mode: {mode}]")
            if mode == "object":
                controller.set_velocities({"base_forward": 0,
                                           "base_counterclockwise": 0})

        if mode == "teleop":
            velocities = tp.get_normalized_velocities()
            controller.set_velocities(velocities)
        else:
            prev = selected
            selected = (selected
                        + inp.rising_edge("RB", "m")
                        - inp.rising_edge("LB", "n")) % len(objects)

            if selected != prev:
                print(f"Selected: {objects[selected]}")
```

```
                pose = controller.get_object_pose(objects[selected])
                if pose:
                    print(f"  x={pose['x']:.3f}  y={pose['y']:.3f}  z={
                        pose['z']:.3f}  "
                          f"qw={pose['qw']:.3f}  qx={pose['qx']:.3f}  "
                          f"qy={pose['qy']:.3f}  qz={pose['qz']:.3f}")

            delta, gravity = obj_ctrl.get_delta()
            controller.move_object_by(objects[selected], delta)
            controller.set_object_gravity(objects[selected], gravity)

        time.sleep(1 / 30)
```

Key API calls

- `controller.list_scene_objects()` — parses the active scene XML and returns the names of every body that contains a `<freejoint/>`. The list is computed once at startup; it works identically whether you are using the default scene or a RoboCasa kitchen.
- `inp.rising_edge("t", "START")` — detects the `t` key or the gamepad Start button to toggle modes. Tab was deliberately avoided as MuJoCo reserves it for its own viewer shortcuts.
- `inp.rising_edge("RB", "m")` — returns `True` (coerced to `1`) exactly once when either the gamepad `RB` button or the keyboard `m` key is first pressed. The multi-argument form checks any of the supplied inputs. Subtracting a second `rising_edge` call gives $+1$, -1, or 0 in a single expression, which is used here to advance or retreat the selection index.
- `ObjectControlProvider.get_delta()` — samples all movement and rotation keys and returns a pair `(delta, gravity)` where `delta` is a dict of axis offsets (only non-zero axes are included) and `gravity` is `True`, `False`, or `None` (no change this frame).
- `controller.move_object_by(name, delta)` — applies the delta dict to the named free-joint body. An empty dict is silently ignored, so no guard is needed in the loop.
- `controller.set_object_gravity(name, gravity)` — sets gravity compensation for the body: `False` makes it float, `True` makes it fall. `None` is a no-op.
- `controller.get_object_pose(name)` — returns the current world pose as a dict with keys `x`, `y`, `z`, `qw`, `qx`, `qy`, `qz`. The pose is printed whenever the selection changes so that you can copy the coordinates directly into `scene.xml` to set a new default start position.

Pose printing for scene configuration Whenever you cycle to a different object the terminal prints its current world pose:

```
Selected: android_lego
  x=0.180  y=-0.550  z=0.821  qw=0.999  qx=0.000  qy=0.032  qz=0.000
```

You can use these values directly as the `pos` and `quat` attributes of the body in `scene.xml` to save a preferred start configuration.

3 Adding Custom 3D-Scanned Objects

3.1 The MuJoCo Scanned Objects Dataset

The mujoco_scanned_objects repository by Kevin Zakka contains roughly 1 000 real-world objects captured with photogrammetry. Every object folder provides:

- `model.obj` — the textured 3D mesh (Wavefront OBJ format)
- `texture.png` — the colour texture image
- `model.xml` — a standalone MuJoCo XML fragment showing how the asset is typically loaded, including pre-computed V-HACD collision decomposition meshes (`model_collision_*.obj`)
- `model_collision_*.obj` — fine-grained collision meshes (optional; a single box is sufficient for most use cases)

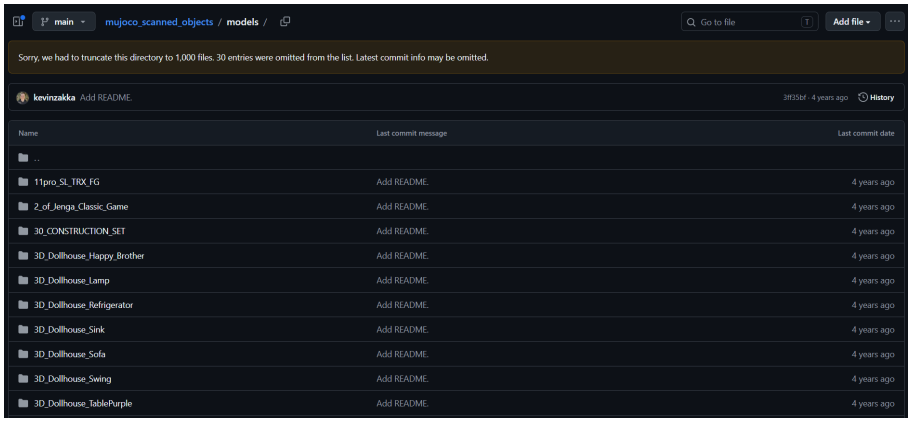


Figure 3: A selection of objects from the mujoco_scanned_objects dataset. Each row corresponds to one folder in the `models/` directory.

3.2 Asset Placement

Assets must be placed under:

```
stretch_mujoco/models/assets/custom_objects/mujoco_scanned_objects/
    models/<ObjectName>/
```

`stretch.xml` sets `assetdir="assets"`, so paths inside `scene.xml` are resolved relative to that directory automatically. Only two files are required from each object folder: `model.obj` and `texture.png`.

3.3 Downloading Assets

Concrete example — Android Lego figure

```
$base = "https://raw.githubusercontent.com/kevinzakka/
    mujoco_scanned_objects/main/models/Android_Lego"
$dest = "stretch_mujoco/models/assets/custom_objects/
    mujoco_scanned_objects/models/Android_Lego"
New-Item -ItemType Directory -Force -Path $dest
Invoke-WebRequest "$base/model.obj"   -OutFile "$dest/model.obj"
Invoke-WebRequest "$base/texture.png" -OutFile "$dest/texture.png"
```

Template for any other object Replace `<ObjectName>` with the exact folder name from the repository (case-sensitive):

```
$obj  = "<ObjectName>"
$base = "https://raw.githubusercontent.com/kevinzakka/
    mujoco_scanned_objects/main/models/$obj"
$dest = "stretch_mujoco/models/assets/custom_objects/
    mujoco_scanned_objects/models/$obj"
New-Item -ItemType Directory -Force -Path $dest
Invoke-WebRequest "$base/model.obj"   -OutFile "$dest/model.obj"
Invoke-WebRequest "$base/texture.png" -OutFile "$dest/texture.png"
```

You can browse available object names at: `https://github.com/kevinzakka/mujoco_scanned_objects/tree/main/models`

3.4 Adding the Object to `scene.xml`

Once the files are in place, edit `stretch_mujoco/models/scene.xml` to declare the assets and add a freejoint body. The structure below is the canonical pattern used for the Android Lego figure.

```
<!-- Inside <asset> -->
<texture name="android_lego_texture" type="2d"
         file="custom_objects/mujoco_scanned_objects/models/
            Android_Lego/texture.png"/>
<material name="android_lego_material" texture="android_lego_texture"/>
<mesh name="android_lego_mesh"
      file="custom_objects/mujoco_scanned_objects/models/Android_Lego/
         model.obj"
      scale="1 1 1"/>

<!-- Inside <worldbody> -->
<body name="android_lego" pos="0.18 -0.55 0.75" gravcomp="1">
  <freejoint/>

  <!-- Visual mesh only (no collision contribution) -->
  <geom name="android_lego_visual" type="mesh"
        mesh="android_lego_mesh" material="android_lego_material"
        contype="0" conaffinity="0" density="0"/>

  <!-- Invisible collision approximation -->
  <geom name="android_lego_collision" type="box"
        pos="0 0 0.045" size="0.028 0.020 0.042"
        mass="0.10" rgba="1 0 0 0" friction="1 0.005 0.0001"/>
</body>
```

Why two geoms? MuJoCo can use any mesh for collision, but fine-grained mesh collision is computationally expensive and can cause instability. The recommended pattern is to keep the visual mesh display-only (`contype="0" conaffinity="0" density="0"`) and add a separate primitive geom (box, cylinder, or sphere) as the physics proxy. Set `rgba="... 0"` on the collision geom to keep it invisible in the viewer and camera feeds.

Collision size The `model.xml` file inside each object folder, while not used directly, gives you the object's bounding information. Use the object control demo (§ 2) to move the object around after adding it—if the collision proxy is obviously wrong (the object sinks through the table or floats above it), adjust the `size` and `pos` values of the collision geom.

`gravcomp="1"` Setting `gravcomp="1"` on the body makes it start in a floating state (gravity fully compensated). This is consistent with the `set_object_gravity` API: calling it with `enabled=False` keeps the object floating, `enabled=True` lets it fall. The default in the demo is floating so that objects stay where you place them until gravity is explicitly enabled.

Freejoint and `list_scene_objects()` Any body with a `<freejoint/>` is automatically discovered by `controller.list_scene_objects()` and immediately controllable via the object control demo without any additional code changes.

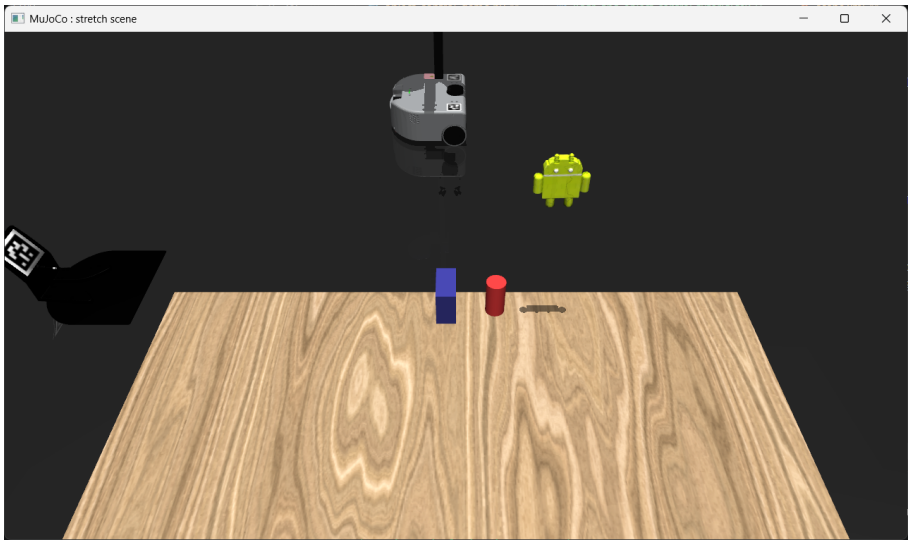


Figure 4: The Android Lego figure placed on the table alongside the existing primitive objects. All three bodies are selectable and movable in the object control demo.

3.5 Adding Objects to a RoboCasa Kitchen

When the simulator is configured to use a RoboCasa kitchen environment (`robocasa.enabled: true` in `stretch_toolkit/sim_config.json`), the scene XML is generated procedurally at startup rather than loaded from a static file. Custom objects are injected into this generated XML through the `custom_objects` array in the same config file.

Configuration Open `stretch_toolkit/sim_config.json` and add entries to the `custom_objects` array inside the `robocasa` block:

```
{
  "robocasa": {
    "enabled": true,
    "task": "PnPCounterToCab",
    "layout": 0,
    "style": 0,
    "custom_objects": [
      {
        "name": "android_lego",
        "obj":  "custom_objects/mujoco_scanned_objects/models/
           Android_Lego/model.obj",
        "texture": "custom_objects/mujoco_scanned_objects/models/
           Android_Lego/texture.png",
        "pos": [0.0, 0.0, 1.0],
        "collision_size": [0.028, 0.020, 0.042],
        "collision_pos":  [0.0, 0.0, 0.045],
```

```
        "mass": 0.10,
        "gravcomp": 1.0
      }
    ]
  }
}
```

Config fields

- `name` — the MuJoCo body name; must be unique within the scene. This is also the name returned by `list_scene_objects()` and accepted by `move_object_by` and `get_object_pose`.
- `obj` / `texture` — paths relative to `stretch_mujoco/models/assets/`. These must point to the same files downloaded in § 3.3.
- `pos` — `[x, y, z]` world position in metres. Because the kitchen layout varies, `[0, 0, 1]` (one metre above the floor) is a safe starting value; adjust after viewing the scene.
- `collision_size` / `collision_pos` — half-extents and local offset of the box collision proxy, in metres. Same meaning as the `size` / `pos` attributes in the `scene.xml` approach.
- `mass` — body mass in kilograms.
- `gravcomp` — `1.0` to start floating, `0.0` to start falling under gravity.

How injection works At startup `stretch_toolkit` reads the config, calls `model_generation_wizard` to produce the kitchen XML string, and then calls `xml_inject_mesh_object` once per entry to splice the asset declarations and freejoint body into the string before passing it to MuJoCo. No manual XML editing is required—only the JSON config entry and the two asset files on disk.



Figure 5: The Android Lego figure spawned inside a RoboCasa kitchen via `sim_config.json`. The object is immediately selectable in the object control demo.